

CHAPTER 4: RESULTS AND DISCUSSION

This chapter outlines the outcomes of the MediConnect pharmacy connection platform developed as a capstone project at the University of Zimbabwe. It covers hardware and software requirements, the testing strategy and module test cases, implementation details for each system component, and screenshots of the deployed web application. The goal is to demonstrate how the AI-powered pharmacy search platform improves medicine access for patients, supports pharmacists in managing inventory and responses, and gives administrators full visibility over platform operations.

4.1 HARDWARE SPECIFICATIONS

This section provides the hardware requirements necessary to run the MediConnect system, covering both the server hosting the backend and client devices used to access the platform.

Table 1 Computer Specifications

Hardware	Technical Specifications	Purpose
Processor	Intel Core i5 / AMD Ryzen 5 or above	Handles Django REST API processing, Daphne ASGI WebSocket connections, and Gemini AI API calls
RAM	8 GB minimum; 16 GB recommended	Supports concurrent execution of Daphne, Django ORM, Django Channels, and database connections under load
Storage	SSD, 50 GB minimum	Fast I/O for the database, prescription image uploads (Django media), and application logs
Network	10 Mbps broadband uplink minimum	Required for real-time WebSocket push to pharmacy clients and outbound Gemini API calls

Table 2 Mobile Device Requirements

Hardware	Technical Specifications	Purpose
Android	Android 8.0 (Oreo) or above	Patient access to medicine search, AI chatbot, and reservation tracking via responsive web

Hardware	Technical Specifications	Purpose
RAM	2 GB or above	Memory handling for browser and PWA
Storage	16 GB or above	App cache and PWA installation data
Network	3G mobile data minimum or Wi-Fi	API calls, WebSocket notifications, and prescription image upload

4.2 SOFTWARE REQUIREMENTS

The system is web-based and mobile-compatible via a Progressive Web App (PWA). It supports all major modern browsers and integrates with the Django REST backend, Django Channels WebSocket layer, and Google Gemini AI services.

Table 3 Supported Browsers (Mobile)

Platform	Browser	Minimum Version
iOS	Safari, Chrome	14+ / 89+
Android	Chrome	89+

Table 4 Supported Browsers (Computer)

Browser	Self-Hosted Minimum Version	Cloud Deployment Minimum Version
Chrome	v89+	v89+
Firefox	v78+	v78+
Safari	v14+	v14+
Edge	v88+	v88+

Table 5 Backend Software Stack (pharmacybackend)

Software / Framework	Version	Role
Python	3.10+	Primary backend language
Django	6.x	Core web framework: ORM, routing, admin site, CSRF middleware

Software / Framework	Version	Role
Django REST Framework	3.x	Serialisers, viewsets, authentication; all endpoints under /api/chatbot/
Django Channels + Daphne	4.x / Latest	ASGI WebSocket layer for real-time pharmacy broadcast; Daphne serves the ASGI application in production
google-generativeai	Latest	Gemini LLM for chatbot and AI report; Gemini Vision for prescription OCR
Pillow / OpenCV / pytesseract	Latest	Prescription image preprocessing pipeline before Gemini Vision submission
pyotp	Latest	TOTP multi-factor authentication for patients and pharmacists
SQLite / PostgreSQL / MongoDB	As configured	Flexible storage: SQLite for dev; PostgreSQL or MongoDB for production

Table 6 Frontend Software Stack (pharmacyfrontend)

Software / Framework	Version	Role
React	19.x	Component-based SPA for patient, pharmacist, and admin portals
Vite	7.x	Build tool producing the static dist/ bundle for deployment
react-router-dom	7.x	Client-side routing across /patient/*, /pharmacy/dashboard, /admin/*
jsPDF + marked	4.x / 18.x	Client-side PDF generation from AI Markdown narrative in admin dashboard
vite-plugin-pwa	1.x	Service worker and web manifest for mobile home-screen (PWA) installation
lucide-react	0.56x	Icon library used across all portal views

4.3 TESTING STRATEGY

The system was tested using a combination of integration testing and system-level acceptance testing to ensure quality, accuracy, and functional correctness across all three user roles.

4.3.1 TEST OBJECTIVES

- Verify that patients can search for medicines, submit requests, and receive ranked pharmacy options via the AI chatbot.
- Confirm that prescription image upload correctly extracts medicine names through the Gemini Vision OCR pipeline.
- Ensure pharmacists can view requests, submit availability responses, manage inventory, and receive real-time WebSocket notifications.
- Validate the MCDA ranking algorithm correctly orders pharmacy responses by price, distance, and availability.
- Verify smooth integration between the Django REST backend, Django Channels WebSocket layer, React SPA frontend, and Gemini AI service.

4.4 TEST RESULTS

Below are summaries of the module test cases conducted for each component of the MediConnect system. Testing was carried out manually using Postman for API-level validation and direct browser interaction for end-to-end flow verification.

Table 7 Login Module

Test Scenario	Verify login functionality for all three user roles (patient, pharmacist, administrator)
Test Case Description	Validate that valid credentials log a user in successfully and redirect to the correct role-specific dashboard
Test Case ID	Login01
Test Data	Patient: patient@mediconnect.zw / ***** Pharmacist: pharm@mediconnect.zw / ***** Admin: admin@mediconnect.zw + Django session CSRF
Test Steps	1. Navigate to /login 2. Select role 3. Enter credentials 4. Click Login
Actual Results	Patient redirected to /patient/dashboard; Pharmacist to /pharmacy/dashboard; Admin to /admin/dashboard with session cookie set
Test Case (Pass/Fail/Not Executed)	Pass

Table 8 Patient Registration Module

Test Scenario	Verify patient signup functionality
Test Case Description	Check that a new patient can register and receive a confirmation

Test Case ID	Register01
Test Data	Email: newpatient@example.com, Password: *****, Role: Patient
Test Steps	1. Navigate to /register 2. Fill in name, email, password 3. Click Register
Actual Results	Account created; patient redirected to login page; POST /api/chatbot/register/patient/ returns HTTP 201
Test Case (Pass/Fail/Not Executed)	Pass

Table 9 Forgot Password / Password Reset Module

Test Scenario	Verify password reset via email OTP
Test Case Description	Ensure a user can request a reset code and set a new password
Test Case ID	ForgotPwd01
Test Data	Email: patient@mediconnect.zw, user_type: patient
Test Steps	1. Navigate to /forgot-password 2. Enter email and user type 3. Receive OTP email 4. Enter code and new password 5. Submit
Actual Results	OTP sent via email_service; new password accepted; account accessible with new credentials
Test Case (Pass/Fail/Not Executed)	Pass

Table 10 AI Chatbot and Symptom Guidance Module

Test Scenario	Verify AI-assisted medicine search via symptom description
Test Case Description	Validate that the Gemini LLM chatbot returns relevant medicine suggestions for a patient symptom query and persists the conversation
Test Case ID	Chat01
Test Data	Input: "I have a headache and mild fever" Session: chatbot_session_id auto-generated
Test Steps	1. Log in as patient 2. Navigate to /patient/ai-assistant 3. Type symptom description 4. Send message

Actual Results	Gemini API returns medicine suggestions with safety advisory; ChatConversation and ChatMessage records persisted; response displayed in chat UI
Test Case (Pass/Fail/Not Executed)	Pass

Table 11 Prescription Upload Module

Test Scenario	Verify prescription image upload and medicine name extraction
Test Case Description	Test that a patient can upload a prescription image and receive extracted medicine names in the chat context
Test Case ID	Presc01
Test Data	File: prescription_sample.jpg (printed text) Active session_id present
Test Steps	1. Log in as patient 2. Open AI assistant 3. Click upload prescription 4. Select image file 5. Submit
Actual Results	Pillow/OpenCV preprocesses image; Gemini Vision extracts medicine names; names injected into ChatConversation context; patient sees extracted list in chat
Test Case (Pass/Fail/Not Executed)	Pass

Table 12 Medicine Request and Pharmacy Broadcast Module

Test Scenario	Verify that a confirmed medicine request is broadcast to all verified pharmacies via WebSocket
Test Case Description	Validate the creation of a MedicineRequest record and real-time notification to pharmacist sessions
Test Case ID	Request01
Test Data	Medicine: Amoxicillin 500mg Patient location: Harare CBD
Test Steps	1. Patient confirms medicine request via chatbot 2. System creates MedicineRequest 3. Monitor pharmacist WebSocket endpoint
Actual Results	MedicineRequest persisted; all verified pharmacies within broadcast radius receive real-time WebSocket push notification

Test Case (Pass/Fail/Not Executed)	Pass
---	------

Table 13 Pharmacist Response and Inventory Module

Test Scenario	Verify pharmacist can view requests, submit a response, and manage inventory
Test Case Description	Validate response submission stores availability and price, notifies the patient, and that bulk inventory update works correctly
Test Case ID	Pharma01
Test Data	Medicine: Amoxicillin 500mg In stock: true Price: ZWL 480.00
Test Steps	1. Log in as pharmacist 2. Navigate to /pharmacy/dashboard 3. Select incoming request 4. Enter availability and price 5. Submit response
Actual Results	PharmacyResponse record created; price_at_reservation captured; patient receives WebSocket notification; inventory CSV export reflects updated values
Test Case (Pass/Fail/Not Executed)	Pass

Table 14 MCDA Pharmacy Ranking Module

Test Scenario	Verify multi-criteria ranking of pharmacy responses for patient decision support
Test Case Description	Validate that multiple PharmacyResponse records are ranked by the MCDA algorithm using price, distance, and availability
Test Case ID	Rank01
Test Data	3 pharmacy responses: prices ZWL 450 / 600 / 520; distances 0.3 km / 1.2 km / 0.7 km
Test Steps	1. Patient submits request 2. Three pharmacies respond 3. Patient views ranked list via GET /api/chatbot/request/<uuid>/ranked/
Actual Results	MCDA service returns list ordered by composite score; lowest-cost nearest option ranks first; confirmed-stock pharmacy receives ranking boost

Test Case (Pass/Fail/Not Executed)	Pass
---	------

Table 15 Reservation Lifecycle Module

Test Scenario	Verify reservation creation, price snapshot integrity, pickup confirmation, and automatic expiry
Test Case Description	Validate the full reservation lifecycle from booking through pickup confirmation or expiry
Test Case ID	Res01
Test Data	Patient selects Pharmacy B at ZWL 480.00 Amoxicillin 500mg x2 24-hour pickup window
Test Steps	1. Patient confirms reservation 2. Pharmacist marks picked_up 3. System calls record_purchase 4. In separate test: simulate 25-hour elapsed time
Actual Results	Reservation created with price_at_reservation = ZWL 480.00; on pickup: inventory decrements by 2, status = completed; on timeout: status = expired, hold released
Test Case (Pass/Fail/Not Executed)	Pass

Table 16 Admin Dashboard Module

Test Scenario	Verify admin dashboard aggregation and operational statistics
Test Case Description	Validate that the admin dashboard returns correct platform statistics and that the background full-hydrate completes without error
Test Case ID	Admin01
Test Data	Admin session active Date range: last 7 days
Test Steps	1. Log in as administrator via /admin/login 2. Bootstrap CSRF from /api/chatbot/admin/csrf/ 3. Navigate to /admin/dashboard
Actual Results	Dashboard displays total requests, active reservations, verified pharmacy count, top-searched medicines, and SLA metrics; background full-hydrate fetch completes
Test Case (Pass/Fail/Not Executed)	Pass

Table 17 AI Report Generation Module

Test Scenario	Verify admin AI narrative report generation, Markdown rendering, and PDF export
Test Case Description	Validate full report pipeline: backend Gemini API call → Markdown response → marked.js parsing → jsPDF client-side download
Test Case ID	Report01
Test Data	Admin selects report for the last 30 days
Test Steps	1. Navigate to /admin/control-center 2. Select report date range 3. Click Generate Report 4. Click Download PDF
Actual Results	Gemini API returns Markdown narrative; marked.js parses response; admin downloads a themed PDF report via jsPDF without a server round-trip
Test Case (Pass/Fail/Not Executed)	Pass

Table 18 Chatbot Safety Policy Module

Test Scenario	Verify admin can update chatbot safety settings dynamically without redeployment
Test Case Description	Validate that PlatformAdminSettings updates take effect on subsequent chatbot sessions
Test Case ID	Policy01
Test Data	Setting: escalation_threshold updated to 3 CSRF token from sessionStorage
Test Steps	1. Admin navigates to chatbot policy settings 2. Updates safety threshold 3. Saves with X-CSRFToken header
Actual Results	PlatformAdminSettings record updated; new policy applied to next chatbot session; AdminAuditLog entry created
Test Case (Pass/Fail/Not Executed)	Pass

4.5 IMPLEMENTATION OF THE DESIGN (CODING AND SIMULATION)

The implemented MediConnect system consists of five core components that interact cohesively to deliver the pharmacy connection experience. The architecture is designed across

two repositories — pharmacybackend (Django REST API) and pharmacyfrontend (React SPA) — with the AI subsystem running entirely on the backend and the PDF export handled client-side. Below is a description of each major component as implemented.

COMPONENT 1: AI CHATBOT AND SYMPTOM GUIDANCE (CHATBOT.JSX + CHATBOT/SERVICES.PY)

The first component is an AI-powered chatbot that guides patients through medicine discovery. Patients describe their symptoms in natural language through the Chatbot.jsx component, which sends the query to POST /api/chatbot/chat/ on the backend. The chatbot.services module invokes the Google Gemini API via the google-generativeai library to generate contextually relevant medicine suggestions accompanied by a safety advisory. All messages are persisted in ChatConversation and ChatMessage models, and the chatbot_session_id and chatbot_conversation_id stored in the patient's localStorage are passed as query parameters to maintain multi-turn context. This component directly addresses the patient's inability to interpret symptoms without clinical guidance.

COMPONENT 2: PRESCRIPTION IMAGE UPLOAD AND OCR (CHATBOT/SERVICES.PY + GEMINI VISION)

The second component handles the upload and analysis of prescription images. Patients upload a prescription photograph through the patient portal; the backend passes the image through a Pillow and OpenCV preprocessing pipeline before submitting it to the Gemini Vision model via POST /api/chatbot/upload-prescription/. The model extracts medicine names which are injected into the active ChatConversation context, allowing the chatbot to use them in subsequent ranking requests. A session_type field on ChatConversation ensures that prescription metadata is scoped to its session and cannot contaminate a separate symptom-only chat. For low-quality or handwritten prescriptions, a quality_warning flag is returned and displayed to the patient.

COMPONENT 3: MEDICINE REQUEST BROADCAST AND MCDA RANKING (CHATBOT/SERVICES.PY)

The third component covers the core request-and-ranking flow. When a patient confirms a medicine request, a MedicineRequest record is created and broadcast in real time to all verified pharmacies in the patient's area via Django Channels WebSocket. Pharmacists see the request in their PharmacyDashboard.jsx and submit a PharmacyResponse with availability and price. The MCDA ranking engine in chatbot.services scores all responses using price, distance, availability, and verified status as criteria. Where a pharmacy also holds a live PharmacyInventory entry for the requested medicine, it receives a confirmed-stock indicator and ranking boost. The ranked list is returned to the patient via GET /api/chatbot/request/<uuid>/ranked/ for selection.

COMPONENT 4: RESERVATION MANAGEMENT (RESERVATION MODEL + RECORD_PURCHASE ENDPOINT)

The fourth component manages the reservation lifecycle. Once a patient selects a pharmacy from the ranked listing, a Reservation record is created via POST /api/chatbot/reserve/ with the price_at_reservation field copied from the originating PharmacyResponse — ensuring the quoted price is preserved even if the pharmacy later updates its inventory. The reservation carries a time-bounded pickup window; if the patient does not collect within the window, the reservation expires automatically and the inventory hold is released. On collection, the pharmacist marks the reservation as picked_up, which triggers POST /api/chatbot/record-purchase/ to decrement PharmacyInventory by the reserved quantity. Commercial settlement is handled off-platform (in-store).

COMPONENT 5: ADMIN ANALYTICS, REPORTING, AND GOVERNANCE (ADMIN_ANALYTICS.PY + ADMIN DASHBOARD.JSX)

The fifth component covers platform administration. Administrators access aggregated statistics, SLA metrics, and pharmacy verification queues through AdminDashboard.jsx and the AdminControlCenter pages. AI-generated narrative reports are produced by sending aggregate platform data to the Gemini API; the Markdown response is parsed by the marked library in AdminDashboard.jsx and converted to a downloadable themed PDF by jsPDF — entirely client-side, without a server-side PDF renderer. Platform governance features include dynamic chatbot safety settings via PlatformAdminSettings, a ChatbotSafetyReview queue, and a full AdminAuditLog for all mutating admin operations.

4.6 SYSTEM ARCHITECTURE AND INTEGRATION

All five components communicate within a unified interface across two repositories. The pharmacybackend Django project exposes all business logic through the /api/chatbot/ route group in chatbot/urls.py and chatbot/views.py. The pharmacyfrontend React SPA (built with Vite 7) communicates with the backend exclusively through REST calls via the src/utils/api.js module and a Django Channels WebSocket connection. Role-based session management partitions the three user contexts: patients use JWT tokens stored in localStorage (patient, chatbot_session_id, chatbot_conversation_id, token); pharmacists use a parallel localStorage structure (pharmacist, pharmacist_id, pharmacy_id, token); administrators use Django session cookies combined with a CSRF token bootstrapped from /api/chatbot/admin/csrf/ before any mutating request.

The Daphne ASGI server runs pharmacybackend.asgi:application in production, handling both HTTP and WebSocket connections concurrently. The frontend dist/ bundle is deployed to a static host (Vercel or Netlify) with VITE_API_URL set to the backend origin at build time. CORS policy is managed by django-cors-headers. The complete data model comprises: ChatConversation, ChatMessage (AI layer); MedicineRequest, PharmacyResponse, ranking snapshots (request/response layer); Pharmacy, Pharmacist, PharmacyInventory,

PharmacyRating (directory layer); PatientProfile, SavedMedicine, PatientNotification, Reservation (patient layer); and PlatformAdminSettings, ChatbotSafetyReview, PharmacySettings, AdminAuditLog (governance layer).

4.7 WEB DASHBOARD SCREENSHOTS

The following figures present screenshots of the deployed MediConnect web application across the public pages, patient portal, pharmacist dashboard, and administrator control centre.

[PLACEHOLDER: Insert screenshot – MediConnect Landing Page (LandingPage.jsx)]

Figure 1 MediConnect Landing Page (LandingPage.jsx)

[PLACEHOLDER: Insert screenshot – Patient Login Page (/login — patient role selected)]

Figure 2 Patient Login Page (/login — patient role selected)

[PLACEHOLDER: Insert screenshot – Patient Registration Page (/register)]

Figure 3 Patient Registration Page (/register)

[PLACEHOLDER: Insert screenshot – Forgot Password Page (/forgot-password)]

Figure 4 Forgot Password Page (/forgot-password)

4.7.1 PATIENT PORTAL

[PLACEHOLDER: Insert screenshot – Patient Dashboard (/patient/dashboard — showing active requests and notifications)]

Figure 5 Patient Dashboard (/patient/dashboard — showing active requests and notifications)

[PLACEHOLDER: Insert screenshot – AI Chatbot — Symptom Input and Medicine Suggestions (/patient/ai-assistant)]

Figure 6 AI Chatbot — Symptom Input and Medicine Suggestions (/patient/ai-assistant)

[PLACEHOLDER: Insert screenshot – Prescription Image Upload — Medicine Names Extracted from Image]

Figure 7 Prescription Image Upload — Medicine Names Extracted from Image

[PLACEHOLDER: Insert screenshot – Ranked Pharmacy Options — MCDA Results View]

Figure 8 Ranked Pharmacy Options — MCDA Results View

[PLACEHOLDER: Insert screenshot – Patient Reservation Confirmation Page]

Figure 9 Patient Reservation Confirmation Page

[PLACEHOLDER: Insert screenshot – Patient Request History (/patient/history)]

Figure 10 Patient Request History (/patient/history)

[PLACEHOLDER: Insert screenshot – Saved Medicines Page (/patient/saved)]

Figure 11 Saved Medicines Page (/patient/saved)

[PLACEHOLDER: Insert screenshot – Patient Notifications Page (/patient/notifications)]

Figure 12 Patient Notifications Page (/patient/notifications)

[PLACEHOLDER: Insert screenshot – Patient Profile and Settings (/patient/profile, /patient/settings)]

Figure 13 Patient Profile and Settings (/patient/profile, /patient/settings)

4.7.2 PHARMACIST DASHBOARD

[PLACEHOLDER: Insert screenshot – Pharmacist Dashboard — Incoming Medicine Requests (/pharmacy/dashboard)]

Figure 14 Pharmacist Dashboard — Incoming Medicine Requests (/pharmacy/dashboard)

[PLACEHOLDER: Insert screenshot – Pharmacist Response Submission — Availability and Price Entry]

Figure 15 Pharmacist Response Submission — Availability and Price Entry

[PLACEHOLDER: Insert screenshot – Pharmacy Inventory Management — Stock Levels and Bulk Update]

Figure 16 Pharmacy Inventory Management — Stock Levels and Bulk Update

[PLACEHOLDER: Insert screenshot – Pharmacy Inventory CSV Export]

Figure 17 Pharmacy Inventory CSV Export

[PLACEHOLDER: Insert screenshot – Pharmacist Reservations — Active and Completed]

Figure 18 Pharmacist Reservations — Active and Completed

[PLACEHOLDER: Insert screenshot – Pharmacist Ranking Summary — Performance Leaderboard]

Figure 19 Pharmacist Ranking Summary — Performance Leaderboard

[PLACEHOLDER: Insert screenshot – Pharmacist Settings and 2FA Setup]

Figure 20 Pharmacist Settings and 2FA Setup

4.7.3 ADMINISTRATOR CONTROL CENTRE

[PLACEHOLDER: Insert screenshot – Admin Dashboard — Platform Statistics and MediBot Overview (/admin/dashboard)]

Figure 21 Admin Dashboard — Platform Statistics and MediBot Overview (/admin/dashboard)

[PLACEHOLDER: Insert screenshot – Admin Control Centre — Heatmap, SLA, and Search Volume Analytics (/admin/control-center)]

Figure 22 Admin Control Centre — Heatmap, SLA, and Search Volume Analytics (/admin/control-center)

[PLACEHOLDER: Insert screenshot – Pharmacy Management — Verification Queue and Pharmacy List (/admin/pharmacies)]

Figure 23 Pharmacy Management — Verification Queue and Pharmacy List (/admin/pharmacies)

[PLACEHOLDER: Insert screenshot – Pharmacist Management Page (/admin/pharmacists)]

Figure 24 Pharmacist Management Page (/admin/pharmacists)

[PLACEHOLDER: Insert screenshot – Request and Reservation Status Overview (/admin/requests)]

Figure 25 Request and Reservation Status Overview (/admin/requests)

[PLACEHOLDER: Insert screenshot – AI Narrative Report — Markdown Rendered in Dashboard]

Figure 26 AI Narrative Report — Markdown Rendered in Dashboard

[PLACEHOLDER: Insert screenshot – AI Report PDF Download (jsPDF + marked client-side export)]

Figure 27 AI Report PDF Download (jsPDF + marked client-side export)

[PLACEHOLDER: Insert screenshot – Chatbot Safety Policy Settings — Dynamic PlatformAdminSettings Update]

Figure 28 Chatbot Safety Policy Settings — Dynamic PlatformAdminSettings Update

[PLACEHOLDER: Insert screenshot – Admin Audit Log — Full Activity Trace]

Figure 29 Admin Audit Log — Full Activity Trace

[PLACEHOLDER: Insert screenshot – Patient Session Lookup Tool (/admin/patients/:sessionId)]

Figure 30 Patient Session Lookup Tool (/admin/patients/:sessionId)